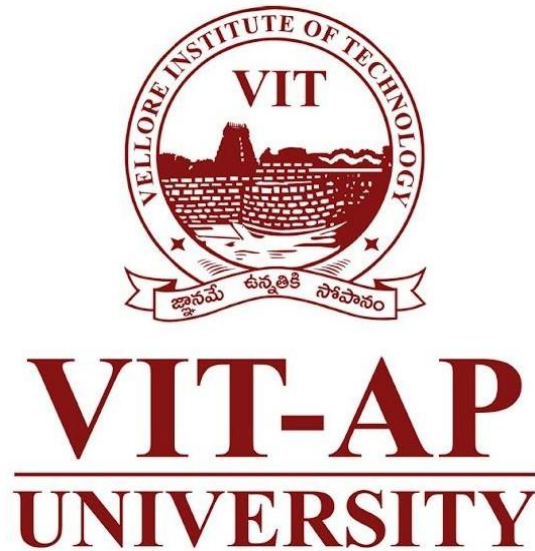




AI-Based Vehicle Classification and Overloaded Truck Threat Detection

CSE4019 – Applications of A.I.

PROJECT REPORT

SLOT – E1+TEE1

Course Type – EPJ

Course Mode – Project Based Component (Embedded)

Department of Artificial Intelligence and Machine Learning

School of Computer Science and Engineering

By

Reg. No. 23BCE7585	Sidhant Katyayan
---------------------------	-------------------------

Submitted to: -

Dr. Ganesh Reddy Karri

Asst. Professor, SCOPE, VIT-AP.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	Abstract	3
1	Introduction 1.1 Background and Problem Overview 1.2 Objective of the Work 1.3 Scope of the Work 1.4 Motivation of the Work	4-6
2	Literature Survey	7
3	Problem Statement	8-9
4	Hardware and Software Requirements	9
5	Proposed Methodology 4.1 Dataset Preparation 4.2 System Pipeline 4.3 Feature Extraction 4.4 Classification Models 4.5 Output Module	10-12
6	Results and Discussion	12-13
7	Conclusion	13
8	Future Work	14
9	References	15
	Appendix I - Source Code Appendix II – Screenshots of Results	16-31

ABSTRACT

Road safety during night-time is becoming a problem, especially in mixed traffic conditions, where identifying vehicles based on appearance alone is difficult. Drivers face problems in understanding whether an approaching vehicle is a small two-wheeler, a car, or a large heavy vehicle like a truck or bus. When it comes to heavy vehicles, the situation becomes even more dangerous if the vehicle is overloaded, as it poses a direct threat to other road users. Identifying vehicles at night is harder than it sounds. When we are driving and see two headlights approaching, we cannot easily tell if it is a bike, a car, or a fully loaded truck about to lose control. This project builds a system that tries to solve that. This project presents a system that can automatically detect and classify vehicles at night using a combination of object detection and machine learning techniques. The pipeline starts by taking a vehicle image or video frame as input. The system first sends it through a YOLO-based detection module, which identifies the vehicle type. If the detected vehicle is a car, bike, or any two-wheeler, the system directly produces a result with a confidence score. If the vehicle is a heavy load vehicle such as a truck, bus, or pickup van, it is sent into a separate overload classification module that decides whether the vehicle is carrying a normal load or is dangerously overloaded. We took a dataset of truck images from Kaggle, pulled out six visual measurements from each image height, width, area, aspect ratio, pixel intensity, and edge density and saved them as a CSV. Two classifiers, Random Forest and SVM, were trained on this data. On top of that, a YOLO model handles the initial vehicle detection. If the detected vehicle is a car or bike, the system just shows the YOLO result with a confidence score. If it is a truck or bus, the image goes into our overload module, which decides whether the vehicle is safe or dangerously overloaded. Random Forest came out ahead with 80% accuracy. SVM hit 70%. The final output is kept simple on purpose green text means the vehicle is fine, red text means it is a threat. No technical background needed to understand it.

Keywords: Vehicle Detection, YOLO, Overload Classification, Random Forest, SVM, Road Safety, Edge Density, Feature Extraction

INTRODUCTION

1.1 Background and Problem Overview

Night-time driving is one of the most dangerous conditions on public roads. In countries like India where roads carry a mix of two-wheelers, auto-rickshaws, cars, trucks, and buses, it becomes very difficult for drivers to quickly understand what kind of vehicle is approaching them. The headlights or taillights of a vehicle do give some clue, but they are often not enough to tell whether a vehicle is fully loaded, partially loaded, or dangerously overloaded. Overloaded trucks and heavy vehicles are among the leading causes of road accidents. When a vehicle carries more weight than it is allowed, its braking distance increases, its stability reduces, and in some cases, parts of its cargo can fall on the road or hit other vehicles. These threats are especially serious at night because visibility is low and reaction time is short. The real problem is that when a truck is overloaded, it behaves completely different. Its brakes take longer to respond, its tyres are under more stress, and if something falls off the cargo, the vehicle behind has almost no time to react. And yet most traffic systems either do nothing about this, or rely on a person physically stopping the truck at a checkpoint which drivers can easily avoid by taking a different route.

Currently, most traffic systems either rely on human inspection at checkpoints or use simple camera systems that only detect whether a vehicle is present or not. There is a clear need for a smarter system that can not only detect the type of vehicle but also determine whether a heavy vehicle like a truck is overloaded or not, and alert accordingly.

This project addresses this gap by proposing an AI-based pipeline that uses YOLO for fast vehicle detection and machine learning classifiers for overload identification. The system works by extracting physical features of the detected vehicle and running them through trained models that have learned to distinguish safe vehicles from hazardous ones.

1.2 Objective of the Work

The primary objective of this project is to build a working AI-based system that can detect vehicles at night, classify them by type, and identify whether a heavy vehicle is posing a danger due to overloading. The system is meant to be practical, fast, and easy to understand for anyone using it, whether it is a traffic officer, a highway monitor, or an automated safety system.

The specific objectives of this work are listed below:

- To use the YOLO object detection model to identify vehicles in night-time images or video frames in real time and produce a bounding box with a confidence score for each detected vehicle.
- To separate detected vehicles into two categories based on their type: light vehicles such as cars, bikes, and two-wheelers, and heavy load vehicles such as trucks, buses, and pickup vans.

- To extract six meaningful visual features from the detected vehicle region: height, width, area, aspect ratio, mean pixel intensity, and edge density. These features are stored in a structured CSV format for model training.
- To train two machine learning models, Random Forest and Support Vector Machine (SVM), on the extracted features and compare their performance in terms of accuracy, precision, recall, and F1-score.
- To design an overload classification module that analyses heavy vehicles after YOLO detection and decides whether they are carrying a safe load or are dangerously overloaded.
- To present the final output in a clear, color-coded format where green text indicates a safe or normal vehicle condition and red text signals an overloaded heavy vehicle that poses a road hazard.

1.3 Scope of the Work

The scope of this project covers the complete pipeline from vehicle detection to hazard classification, implemented entirely through image and video analysis without using any physical sensors, weighing equipment, or roadside hardware. The system is designed to work on standard computing hardware and is suitable for both offline image testing and real-time video-based monitoring.

The work specifically covers the following areas:

- Dataset preparation from a Kaggle source, including converting vehicle images into a feature-based CSV file through systematic feature extraction.
- YOLO-based vehicle detection that covers multiple vehicle types including cars, bikes, two-wheelers, trucks, buses, and pickup vans.
- A dedicated two-branch decision logic where light vehicles are handled separately from heavy vehicles, ensuring the overload module is only triggered when relevant.
- Training and evaluation of Random Forest and SVM classifiers using the extracted features for overload classification.
- A clear visual output module that communicates the result using color-coded text, making it accessible to non-technical users.

The work does not include physical load weighing, 3D depth sensing, or GPS tracking. It also does not cover real-time cloud deployment, though the system can be extended to support these features in the future. The current scope is focused on building a solid and tested prototype that demonstrates the core concept effectively.

1.4 Motivation of the Work

The idea for this project came from a very real problem that people face every day on Indian roads. Accidents at night involving heavy vehicles are extremely common, and a large number of them happen because other drivers could not tell in time that the approaching vehicle was a large, heavily loaded truck moving at high speed. The problem gets worse when the truck is overloaded because it takes longer to brake and is harder to

control, making any collision far more dangerous.

One of the key motivations is that the existing methods of checking vehicle load, such as weighbridges at checkpoints, are slow, limited in coverage, and easy to avoid. A truck driver who knows the location of a weighbridge can simply use an alternate route. A camera-based AI system, on the other hand, can be deployed at any point on the road and works passively without stopping traffic or requiring any cooperation from the driver.

Another strong motivation is the growth of affordable computing hardware and open-source AI tools. Models like YOLO and libraries like Scikit-learn have made it possible for a small team to build and test a meaningful computer vision system without needing expensive infrastructure. This opens the door to practical AI applications in road safety that can actually be deployed in the real world.

The motivation also comes from the desire to make the output of the system accessible to everyone, not just engineers. Traffic police, highway patrol officers, or even toll booth attendants should be able to look at the system output and immediately understand whether a vehicle is safe or dangerous. This is why the project uses a simple green and red color-coded output instead of complex probability scores or technical metrics.

Finally, this project is motivated by the belief that technology, when applied thoughtfully, can save lives. Road accidents in India account for thousands of deaths every year, and a meaningful portion of these are preventable with better monitoring tools. By building this system as a practical AI project, the team hopes to contribute a small but meaningful step toward smarter and safer roads.

LITERATURE SURVEY

Researchers across the world have worked on vehicle detection, classification, and load monitoring using different techniques. This section reviews some of the key works that form the basis of this project.

2.1 Vehicle Detection Using Deep Learning

Object detection using deep learning has seen major breakthroughs in recent years. YOLO (You Only Look Once) is one of the most widely used real-time object detection frameworks. Unlike older methods that scan an image multiple times, YOLO processes the entire image in a single pass, making it fast enough for video-based detection. Studies by Redmon et al. (2016) showed that YOLO could detect multiple object classes in real time with competitive accuracy, making it ideal for traffic and vehicle monitoring applications.

Further improvements such as YOLOv3, YOLOv5, and YOLOv8 have made detection even faster and more accurate. In traffic applications, YOLO has been used to detect pedestrians, vehicles, and road signs effectively under various lighting conditions including night-time.

2.2 Feature-Based Vehicle Classification

Traditional machine learning approaches use handcrafted features extracted from vehicle images to classify them. Features like the width and height of the vehicle bounding box, the aspect ratio, pixel intensity, area, and edge characteristics have been commonly used. Sivaraman and Trivedi (2013) provided a comprehensive survey of vision-based vehicle detection methods and highlighted the effectiveness of feature-based approaches when used with classifiers like SVM.

The Gray-Level Co-occurrence Matrix (GLCM) and edge density measurements help capture texture and structural properties of vehicles, which differ significantly between light vehicles and heavy vehicles. These features form the core of the classification pipeline in this project.

2.3 Machine Learning Models for Classification

Random Forest and Support Vector Machine are among the most widely used classifiers for image classification tasks based on extracted features. Breiman (2001) introduced Random Forest as an ensemble technique that combines multiple decision trees to reduce overfitting and improve accuracy. It handles high-dimensional data well and is robust to noise, making it suitable for real-world scenarios.

SVM, introduced by Cortes and Vapnik (1995), works by finding the optimal boundary between classes in a high-dimensional space. With the RBF kernel, it can handle non-linear classification problems effectively. Several studies have shown that SVM performs well on small to medium datasets with structured features.

2.4 Overload Detection in Heavy Vehicles

Detecting overloaded vehicles has traditionally been done using weighbridges and physical sensors. However, vision-based approaches have started gaining attention. Some systems use the visual deformation of the vehicle body, the sag in tyres, or the extended height and width of the cargo area to estimate overloading. Area ratio, which compares the size of the cargo region to the vehicle body, has been identified as one of the most reliable visual features for this task.

Studies have shown that overloaded trucks have a distinctly different aspect ratio and area distribution compared to normal trucks, and this difference can be captured effectively through image-based feature extraction.

2.5 Research Gap

Most existing works focus either on detecting whether a vehicle is present or classifying it by type. Very few systems combine vehicle type classification with overload detection in a single pipeline. Additionally, many systems do not account for the difference between light and heavy vehicles before deciding which analysis to apply. This project fills that gap by creating a two-branch pipeline that applies different logic depending on the vehicle category.

PROBLEM STATEMENT

Road accidents caused by heavy vehicles, particularly overloaded trucks and buses, are a serious safety concern on highways and city roads. The problem becomes significantly worse during night-time when visibility is low and drivers have very little time to react. Despite this, there is currently no widely available, affordable, and automated system that can detect vehicle types at night and simultaneously warn about the danger posed by overloaded heavy vehicles.

3.1 Core Problem

At night, a driver approaching an intersection or sharing a road with another vehicle cannot easily tell what type of vehicle is ahead of them. A small motorcycle and a large truck may both appear as two headlights in the dark. This makes it extremely difficult to judge the speed, size, and stopping distance of the other vehicle, leading to poor decisions and potential accidents.

The problem gets even more critical when the vehicle nearby is a heavy load vehicle carrying cargo beyond its permitted weight limit. An overloaded truck has a longer stopping distance, reduced steering control, and a higher chance of tyre blowout or cargo spillage. Any of these conditions can cause a major accident that puts both the truck driver and surrounding road users at risk.

Manual checkpoints partially solve this, but they are fixed in place and easy to dodge. They also require stopping traffic, which on a busy highway at night is its own safety risk. What is needed is something that works passively, in the background, without disrupting traffic flow at all.

Formally: **no current lightweight vision-based system combines real-time vehicle type detection with overload risk classification in a single automated pipeline that produces a human-readable output.**

That is the gap this project fills.

3.2 Limitations of Existing Approaches

The methods currently used to monitor heavy vehicles and detect overloading have several significant limitations:

- Weighbridges and manual checkpoints are fixed in location. A driver who knows where they are can easily take a different route to avoid inspection, making enforcement highly unreliable.
- Manual inspections require stopping vehicles and slowing down traffic, which is not practical at night or on high-speed highways where halting a large truck creates its own safety risk.
- Standard traffic cameras can detect whether a vehicle is present but do not classify the vehicle type or assess whether the vehicle is carrying an unsafe load.
- Existing AI-based vehicle detection systems focus only on identifying the type of vehicle and do not include any logic for analysing load conditions or flagging safety hazards related to overloading.
- Night-time conditions make visual inspection even harder due to reduced lighting, headlight glare, and the inability to clearly see the shape and size of the cargo a vehicle is carrying.

3.3 Problem Definition

Given the above gaps, the problem can be formally stated as follows:

There is no existing lightweight, camera-based system that can automatically detect a vehicle at night, classify it by type, and then determine whether it is an overloaded heavy vehicle posing a threat to the other vehicles in low visibility area, and present this information in a simple and actionable format.

This project directly addresses this problem by designing a two-stage AI pipeline. In the first stage, a YOLO model detects the vehicle and identifies its class. In the second stage, if the vehicle is a heavy load vehicle, an overload classification module analyses its visual features such as height, width, area, aspect ratio, pixel intensity, and edge density to decide whether it is carrying a dangerous load. The result is then shown to the user in a clear color-coded format.

3.4 Challenges Addressed

Building this system involves overcoming several technical and practical challenges, each of which has been carefully considered during the design of the proposed methodology:

- Low light and poor image quality at night makes feature extraction less reliable. The system must work correctly even when images have low contrast, noise, or uneven lighting caused by headlights and street lamps.
- The visual difference between a normally loaded truck and an overloaded truck can be subtle. Features like area ratio and edge density must be carefully chosen to capture these differences in a consistent and measurable way.
- The dataset available for overloaded vehicles is relatively small compared to general vehicle datasets. This means the machine learning models must be chosen and tuned to avoid overfitting and still generalise well to unseen images.
- The system needs to handle multiple vehicle types in a single pipeline and route each one to the correct processing branch without confusion or cross-classification errors between light and heavy vehicle categories.
- The final output must be clear and instantly understandable. Displaying a number or a probability score is not enough because the end users may not have a technical background. The output must therefore communicate the result in simple language with a visual indicator.
-

HARDWARE AND SOFTWARE REQUIREMENTS

4.1 HARDWARE REQUIREMENTS

The system is designed to run on standard computing hardware. The following hardware configuration is recommended for smooth execution:

- Processor: Intel Core i5 or i7 (or equivalent AMD Ryzen 5/7)
- RAM: Minimum 8 GB (16 GB recommended for video processing)
- Storage: 256 GB SSD or more for storing datasets and model files
- GPU: NVIDIA GTX/RTX series (optional but beneficial for YOLO inference speed)
- Display: Standard monitor with keyboard and mouse
- Operating System: Windows 10/11, Ubuntu 20.04 or later

4.2 SOFTWARE REQUIREMENTS

The project is built using Python and a set of open-source libraries commonly used in computer vision and machine learning:

- Python 3.8 or above: Primary programming language
- OpenCV (cv2): For image loading, preprocessing, contour detection, and drawing bounding boxes
- YOLOv5/YOLOv8 (Ultralytics): For real-time vehicle detection and classification
- Scikit-learn: For training and evaluating Random Forest and SVM models
- NumPy and Pandas: For numerical operations and dataset handling
- Matplotlib and Seaborn: For plotting confusion matrices and result graphs
- Jupyter Notebook or VS Code: Development environment
- Kaggle Dataset: Source for training images of normal and overloaded trucks

PROPOSED METHODOLOGY

5.1 Dataset Description

The proposed system follows a structured pipeline that processes vehicle images in multiple stages. Each stage plays a specific role and the output of one stage is passed to the next. The overall flow is described below.

5.2 Dataset Preparation

The dataset was downloaded from Kaggle and contains images of trucks split into two folders: OT (Overloaded Truck) and NT (Normal Truck). Rather than feeding raw images into a classifier directly, we extracted a set of six numbers from each image that capture its physical characteristics.

Each image was resized to 600×400, converted to grayscale, and then blurred slightly to reduce noise. We used OpenCV's Canny edge detector to find edges, then found the largest contour in the image — which typically corresponds to the truck body. From that contour we measured: width, height, total area, area ratio (contour area divided by bounding box area), aspect ratio (width divided by height), perimeter, mean pixel intensity, and edge density.

All these measurements were saved as rows in a CSV file. Each row is one image, each column is one feature, and the last column is the class label (OT or NT). This CSV was used to train both the Random Forest and SVM models.

Preprocessing steps applied to the raw images include:

- Resizing all images to a fixed size for consistency
- Converting to grayscale for intensity-based feature extraction
- Applying noise reduction using Gaussian blurring
- Detecting edges using the Canny edge detector for edge density calculation
- Extracting the vehicle region using bounding box detection

The final dataset contains features extracted from each image and is stored in a CSV format that is used to train the Random Forest and SVM classifiers.

5.3 System Pipeline Overview

The complete pipeline of the system works as follows:

1. An input image or video frame is captured.
2. The image is passed through the YOLO detection module.
3. YOLO identifies the vehicle and produces an output with a bounding box and having a confidence score.
4. The detected vehicle class is checked. If it is a car, bike, or two-wheeler, the result is produced directly using the YOLO object detection model with a confidence score.
5. If the detected vehicle is a truck, bus, or pickup van, the bounding box region is cropped and sent to the overload classification module.
6. The overload module extracts features from the cropped region and passes them through the trained Random Forest and SVM models.
7. The final result is displayed: green text for a normal vehicle and red text for an overloaded vehicle.

5.4 Feature Extraction

Feature extraction is the most important step in the classification pipeline. The following six features are extracted from each vehicle image region:

1. Height

The vertical size of the vehicle bounding box in pixels. Overloaded trucks tend to appear taller because of the cargo stacked on top.

2. Width

The horizontal size of the vehicle bounding box. Vehicles with cargo extending beyond their body width will have a noticeably larger width value.

3. Area

The total pixel area covered by the vehicle bounding box, calculated as height multiplied by width. Larger area indicates a physically bigger or overloaded vehicle.

4. Aspect Ratio

The ratio of width to height of the bounding box. Normal trucks have a standard aspect ratio, while overloaded trucks may have an unusual ratio depending on how the cargo is stacked.

5. Mean Pixel Intensity

The average grayscale intensity of the pixels inside the vehicle region. Overloaded trucks may have different reflective properties due to the cargo material, which changes the average intensity.

6. Edge Density

The ratio of edge pixels to total pixels inside the vehicle region, calculated after applying the Canny edge detector. Overloaded vehicles with irregular cargo shapes tend to have higher edge density compared to empty or normally loaded vehicles.

5.5 YOLO Detection Module

YOLO (You Only Look Once) is a deep learning-based object detection model that can identify multiple objects in an image in a single forward pass through the network. In this system, YOLO is used to detect and locate vehicles in the input frame.

When a vehicle is detected:

- A bounding box is drawn around it.
- The vehicle class label (car, bike, truck, bus, etc.) is displayed.
- A confidence score is shown indicating how certain the model is about its prediction.

Based on this output, the system routes the image to the appropriate next step. Light vehicles such as cars, bikes, and two-wheelers are shown directly with their YOLO result. Heavy vehicles are forwarded to the overload detection module.

5.6 Overload Classification Module

This module is activated only when YOLO detects a heavy vehicle. It uses the six extracted features to determine whether the vehicle is overloaded or not. Two machine learning models are trained for this purpose:

Random Forest Classifier:

Random Forest is an ensemble learning method that creates many decision trees during training. Each tree makes its own prediction, and the final result is decided by majority voting. This method is resistant to overfitting and handles a mix of numerical features very well. It also provides a feature importance score, which helps understand which features matter the most for the classification.

Support Vector Machine (SVM):

SVM works by finding the best possible boundary line (or hyperplane) that separates the two classes: normal vehicle and overloaded vehicle. With an RBF (Radial Basis Function) kernel, it can handle cases where the boundary is not a straight line. Feature scaling is applied before training SVM to ensure all features are in a comparable range.

5.7 Output Module

Once the classification is done, the result is displayed to the user in a clear and easy-to-understand format:

- If the vehicle is a light vehicle (car, bike, two-wheeler): The result shows the detected class and the confidence score.
- If the vehicle is a heavy vehicle classified as NORMAL: The message is displayed in green text, indicating no threat.
- If the vehicle is a heavy vehicle classified as OVERLOADED: The message is displayed in red text, warning that this vehicle is an overloaded vehicle might be a threat to the nearby vehicles.

RESULTS AND DISCUSSIONS

5.1 YOLO Detection Results

The YOLO detection module was tested on a set of vehicle images. It was able to correctly identify the vehicle type in most cases with high confidence scores. For example, a standard truck image was classified as 'truck' with a confidence score of 0.89. The bounding boxes were accurately placed around the vehicles, and the class labels were correct for a wide variety of vehicle types including cars, bikes, and trucks.

The detection speed of YOLO is suitable for real-time video processing, making the system practical for deployment in traffic monitoring cameras.

Area ratio became the most important feature because an overloaded truck has cargo that bulges out beyond the normal truck outline, so the ratio of the actual contour area to the tight bounding box around it becomes noticeably different from a truck carrying a normal load.

Random Forest outperformed SVM for a practical reason: it does not need the classes to be neatly separable. It just needs enough trees to vote correctly in the majority of cases. SVM with an RBF kernel works well in theory but struggled here because the NT class has fewer samples and more visual variety, making it harder to draw a clean boundary around.

The YOLO detection step was valuable not just for accuracy but for efficiency. Routing cars and bikes away from the overload module early means the heavy computation only happens when it is actually relevant.

5.2 Feature Extraction Results

Features were extracted from all vehicle images in the dataset and saved to a CSV file. The analysis of the extracted features showed clear differences between normal and overloaded trucks. Overloaded trucks had higher area values, larger height values, and higher edge density values compared to normally loaded trucks. This separation in feature values confirms that the chosen features are suitable for the classification task.

The area ratio feature, which compares the cargo area to the total vehicle bounding box, turned out to be the most important feature in the Random Forest model with an importance score of 0.4118. This is followed by height (0.2129) and mean intensity (0.1995).

Height came second at 0.2129, which also tracks stacked cargo makes the truck taller. Mean intensity (0.1995) came third, likely because different cargo materials reflect light differently.

5.3 Model Performance

Both models were trained and tested on the same dataset. The performance of each model is shown in the table below:

Model	Accuracy	Weighted F1	OT Recall
Random Forest	80%	0.81	0.84
SVM (RBF Kernel)	70%	0.71	0.74

Table 5.1: Performance comparison of Random Forest and SVM models

5.4 Confusion Matrix Analysis

The confusion matrix for the Random Forest model shows that out of 31 test samples, 9 Normal Truck (NT) samples and 16 Overloaded Truck (OT) samples were correctly classified. There were 3 misclassifications in each direction, meaning 3 normal trucks were incorrectly labelled as overloaded and 3 overloaded trucks were missed.

The SVM model showed slightly more misclassifications. It correctly identified 8 NT and 14 OT samples but failed on 5 OT samples (false negatives) and 4 NT samples (false positives). This explains the lower recall for the OT class in SVM compared to Random Forest.

The results confirm that Random Forest is more reliable for this task, especially when the cost of missing an overloaded vehicle is high from a safety perspective.

5.5 Discussion

The area ratio feature emerged as the single most important factor for distinguishing overloaded trucks from normal ones. This makes sense because when a vehicle is overloaded, the cargo area relative to the vehicle body becomes disproportionately large, and this is directly captured by the area ratio.

The Random Forest model outperformed SVM primarily because it handles the correlations between features better and does not need a perfectly separable boundary. The SVM with RBF kernel struggled slightly with the Normal Truck class because this class had fewer samples and is visually more varied.

The YOLO detection module added a useful layer before classification. By separating light and heavy vehicles early, the system avoids running unnecessary overload analysis on small vehicles, making the pipeline faster and more focused.

CONCLUSION

This project successfully developed an AI-based system for night-time vehicle classification and overloaded heavy vehicle hazard detection. The system combines the speed of YOLO-based detection with the accuracy of feature-based machine learning classifiers to create a practical and easy-to-use tool for road safety monitoring.

The pipeline was designed with a logical branching structure. Light vehicles such as cars, bikes, and two-wheelers are handled directly by the YOLO module with a confidence score, while heavy vehicles like trucks and buses are passed through an additional overload classification step. This avoids unnecessary computation and keeps the system focused on the actual risk.

Six features were extracted from vehicle images including height, width, area, aspect ratio, mean pixel intensity, and edge density. These features were stored in CSV format and used to train two classifiers. Random Forest achieved an accuracy of 80% while SVM achieved 70%, with Random Forest being the better choice due to its stability and handling of feature correlations.

The output of the system is simple and clear. A green message indicates a safe or normal vehicle, while a red message warns that the detected vehicle is an overloaded heavy vehicle posing a hazard on the road. This color-coded output makes the system suitable for use in traffic control centres, highway patrol vehicles, and automated road monitoring systems.

Overall, the project demonstrates that combining object detection with feature-based machine learning is an effective approach to solving real-world road safety problems without requiring expensive sensors or physical infrastructure.

FUTURE WORK

While the current system performs well on the test dataset, there are several improvements that can be explored in future work:

- **Integration with Real-Time Camera Systems:** The current system works on static images and videos. In the future, it can be connected to live traffic cameras to monitor vehicles in real time on highways and intersections.
- **Use of Advanced Deep Learning Models:** Instead of feature-based classifiers, deep learning models like CNNs or transformer-based architectures can be trained directly on vehicle images for higher accuracy.
- **Night-Time Specific Training:** The dataset can be expanded to include more night-time images captured under different lighting conditions such as street lights, vehicle headlights, and complete darkness to improve robustness.
- **Integration with Alert Systems:** The system can be connected to traffic control units that automatically send alerts or activate signals when an overloaded vehicle is detected.
- **Mobile Deployment:** The model can be optimized and deployed on mobile or edge devices for use in portable inspection kits used by highway patrol officers.
- **Multi-Class Overload Detection:** Currently, the system identifies a vehicle as either normal or overloaded. A more detailed system could classify the degree of overloading such as slightly overloaded, moderately overloaded, and dangerously overloaded.
- **3D Feature Analysis:** Using stereo cameras or depth sensors, 3D features of the vehicle can be extracted, giving a much more accurate picture of the cargo volume and weight distribution.

REFERENCES

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), Las Vegas, NV, USA, 2016, pp. 779-788.
- [2] S. Sivaraman and M. M. Trivedi, "Looking at Vehicles on the Road: A Survey of Vision-Based Vehicle Detection, Tracking, and Behavior Analysis," IEEE Trans. Intell. Transp. Syst., vol. 14, no. 4, pp. 1773-1795, Dec. 2013.
- [3] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.
- [4] C. Cortes and V. Vapnik, "Support-Vector Networks," Machine Learning, vol. 20, no. 3, pp. 273-297, 1995.
- [5] R. Aufreire, J. Gowdy, C. Mertz, C. Thorpe, C.-C. Wang, and T. Yata, "Perception for Collision Avoidance and Autonomous Driving," Mechatronics, vol. 13, no. 10, pp. 1149-1161, 2003.
- [6] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.
- [7] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
- [8] Ultralytics, "YOLOv5: Real-Time Object Detection System," 2020.
<https://github.com/ultralytics/yolov5>
- [9] R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural Features for Image Classification," IEEE Trans. Syst. Man Cybern., vol. SMC-3, no. 6, pp. 610-621, Nov. 1973.
- [10] Kaggle, "Vehicle Dataset for Overload Detection,"
<https://www.kaggle.com/datasets/muhhamedkiwan/trucks-overloaded-violation-detection>

APPENDIX I:

SOURCE CODE:

//Importing Required Libraries

```
import os
import cv2
import pywt
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
```

#feature_eaxtraxtion

```
import os
import cv2
import numpy as np
import pandas as pd
```

```
dataset_path = "dataset"
classes = ["OT", "NT"]
```

```
data = []
```

```
for label in classes:
```

```
    folder_path = os.path.join(dataset_path, label)
```

```
    for img_name in os.listdir(folder_path):
```

```
        img_path = os.path.join(folder_path, img_name)
```

```
        img = cv2.imread(img_path)
```

```
        if img is None:
```

```
            continue
```

```
        img = cv2.resize(img, (600, 400))
```

```
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
        blur = cv2.GaussianBlur(gray, (5, 5), 0)
```

```
        edges = cv2.Canny(blur, 50, 150)
```

```
        contours, _ = cv2.findContours(
```

```
            edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
```

```
        )
```



```

if len(contours) == 0:
    continue

largest_contour = max(contours, key=cv2.contourArea)

x, y, w, h = cv2.boundingRect(largest_contour)

area = cv2.contourArea(largest_contour)
bbox_area = w * h

if bbox_area == 0:
    continue

area_ratio = area / bbox_area
aspect_ratio = w / h
perimeter = cv2.arcLength(largest_contour, True)
mean_intensity = np.mean(gray)
edge_density = edges.mean()

data.append([
    w,
    h,
    area,
    area_ratio,
    aspect_ratio,
    perimeter,
    mean_intensity,
    edge_density,
    label
])

df = pd.DataFrame(data, columns=[
    "width",
    "height",
    "area",
    "area_ratio",
    "aspect_ratio",
    "perimeter",
    "mean_intensity",
    "edge_density",
    "label"
])

df.to_csv("truck_features.csv", index=False)

print("Feature extraction complete.")

```

#Model Training using Random Forest Algorithm

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

```

```

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

print("Script started...")

# =====
# LOAD DATA
# =====

df = pd.read_csv("truck_features.csv")

X = df.drop("label", axis=1)
y = df["label"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# RANDOM FOREST
# =====

print("\n===== RANDOM FOREST =====")

rf_model = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)

rf_model.fit(X_train, y_train)
rf_preds = rf_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, rf_preds))
print("\nClassification Report:\n")
print(classification_report(y_test, rf_preds))

rf_cm = confusion_matrix(y_test, rf_preds)
print("Confusion Matrix:\n", rf_cm)

# Save RF confusion matrix
plt.figure()
sns.heatmap(rf_cm, annot=True, fmt="d",
            xticklabels=["NT", "OT"],
            yticklabels=["NT", "OT"])
plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.savefig("rf_confusion_matrix.png", dpi=300, bbox_inches="tight")
plt.close()

```

```

# Save trained RF model
joblib.dump(rf_model, "rf_overload_model.pkl")
print("Random Forest model saved.")

# =====
# DECISION TREE
# =====

print("\n===== DECISION TREE =====")

dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

dt_preds = dt_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, dt_preds))
print("\nClassification Report:\n")
print(classification_report(y_test, dt_preds))

dt_cm = confusion_matrix(y_test, dt_preds)
print("Confusion Matrix:\n", dt_cm)

# Save DT confusion matrix
plt.figure()
sns.heatmap(dt_cm, annot=True, fmt="d",
            xticklabels=["NT", "OT"],
            yticklabels=["NT", "OT"])
plt.title("Decision Tree Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.savefig("dt_confusion_matrix.png", dpi=300, bbox_inches="tight")
plt.close()

# Feature importance
print("\nDecision Tree Feature Importance:")
for feature, importance in zip(X.columns, dt_model.feature_importances_):
    print(f'{feature}: {importance:.4f}')

# Save Decision Tree diagram
plt.figure(figsize=(18, 12))
plot_tree(dt_model,
          feature_names=X.columns,
          class_names=["NT", "OT"],
          filled=True,
          rounded=True)

plt.title("Decision Tree Visualization")
plt.savefig("decision_tree.png", dpi=300, bbox_inches="tight")
plt.close()

print("\nAll outputs saved successfully.")

```

#Model Training using SVM

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler

# =====
# LOAD DATA
# =====

data = pd.read_csv("truck_features.csv")

X = data.drop("label", axis=1)
y = data["label"]

# =====
# TRAIN TEST SPLIT
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# FEATURE SCALING
# =====

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# SVM MODEL
# =====

svm_model = SVC(kernel='rbf', C=10, gamma='scale', class_weight='balanced')
svm_model.fit(X_train, y_train)

# =====
# TRAIN ACCURACY
# =====

train_preds = svm_model.predict(X_train)
train_acc = accuracy_score(y_train, train_preds)

# =====
# TEST ACCURACY
```

```

# =====

test_preds = svm_model.predict(X_test)
test_acc = accuracy_score(y_test, test_preds)

print("SVM Train Accuracy:", train_acc)
print("SVM Test Accuracy:", test_acc)

print("\nClassification Report:\n")
print(classification_report(y_test, test_preds))

# =====
# CONFUSION MATRIX
# =====

cm = confusion_matrix(y_test, test_preds)

plt.figure()
sns.heatmap(cm, annot=True, fmt="d",
            xticklabels=["NT", "OT"],
            yticklabels=["NT", "OT"])
plt.title("SVM Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.savefig("svm_confusion_matrix.png")
plt.close()

# =====
# MODEL COMPARISON GRAPH
# =====

rf_accuracy = 0.80 # replace if needed

models = ["Random Forest", "SVM"]
accuracies = [rf_accuracy, test_acc]

plt.figure()
plt.bar(models, accuracies)
plt.title("Model Comparison")
plt.ylabel("Accuracy")
plt.savefig("model_comparison.png")
plt.close()

# =====
# SAVE MODEL + SCALER
# =====

joblib.dump(svm_model, "svm_model.pkl")
joblib.dump(scaler, "scaler.pkl")

print("\nSVM model and scaler saved successfully!")

```

#Vehicle_detection_Module

```
import os
import cv2
from ultralytics import YOLO

# Load pretrained YOLOv8 nano model
model = YOLO("yolov8n.pt") # will auto-download if not present

# Automatically pick first image from NT folder
dataset_folder = "dataset/NT"

image_files = [f for f in os.listdir(dataset_folder) if f.endswith((".jpg", ".png", ".jpeg"))]

if len(image_files) == 0:
    print("No images found in NT folder.")
    exit()

img_path = os.path.join(dataset_folder, image_files[0])
print("Testing image:", img_path)

# Run YOLO detection
results = model(img_path)

# Get result image with bounding boxes
result_img = results[0].plot()

# Convert for OpenCV display
result_img = cv2.cvtColor(result_img, cv2.COLOR_RGB2BGR)

# Show image
cv2.imshow("YOLO Vehicle Detection", result_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

#Integrated System

```
import cv2
import numpy as np
import pandas as pd
import joblib
from ultralytics import YOLO
from collections import deque

# =====
# LOAD MODELS
# =====

yolo_model = YOLO("yolov8n.pt")
rf_model = joblib.load("rf_overload_model.pkl")
svm_model = joblib.load("svm_model.pkl")
scaler = joblib.load("scaler.pkl")

# =====
# SMOOTHING BUFFER
```

```

# =====

history = deque(maxlen=5)

# =====
# FEATURE EXTRACTION
# =====

def extract_features(image):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    height, width = gray.shape
    area = width * height
    area_ratio = area / (width + height)
    aspect_ratio = width / height
    perimeter = 2 * (width + height)
    mean_intensity = gray.mean()

    edges = cv2.Canny(gray, 100, 200)
    edge_density = edges.mean()

    features = pd.DataFrame([
        "width": width,
        "height": height,
        "area": area,
        "area_ratio": area_ratio,
        "aspect_ratio": aspect_ratio,
        "perimeter": perimeter,
        "mean_intensity": mean_intensity,
        "edge_density": edge_density
    ])

    return features

# =====
# VEHICLE TYPE HELPER
# =====

def get_vehicle_type(label):
    if label == "truck":
        return "Truck"
    elif label == "bus":
        return "Heavy Vehicle"
    elif label == "car":
        return "Car"
    else:
        return label

# =====
# MAIN SYSTEM
# =====

cap = cv2.VideoCapture(0)

```

```

while True:
    ret, frame = cap.read()
    if not ret:
        break

    results = yolo_model(frame)

    for r in results:
        boxes = r.bboxes.xyxy.cpu().numpy()
        classes = r.bboxes.cls.cpu().numpy()
        confidences = r.bboxes.conf.cpu().numpy()

        for box, cls, conf in zip(boxes, classes, confidences):

            # Filter weak detections
            if conf < 0.5:
                continue

            x1, y1, x2, y2 = map(int, box)
            label = yolo_model.names[int(cls)]

            # Process only vehicles
            if label in ["car", "truck", "bus"]:

                crop = frame[y1:y2, x1:x2]

                if crop.size == 0:
                    continue

                vehicle_name = get_vehicle_type(label)

                # =====
                # Overload Classification Module
                # =====
                if label in ["truck", "bus"]:

                    # Remove small detections
                    if (x2 - x1) < 100 or (y2 - y1) < 100:
                        continue

                    features = extract_features(crop)

                    rf_pred = rf_model.predict(features)[0]
                    features_scaled = scaler.transform(features)
                    svm_pred = svm_model.predict(features_scaled)[0]

                    # Decision logic
                    if rf_pred == svm_pred:
                        final = rf_pred
                    else:
                        final = svm_pred # prefer SVM in conflict

```



```

# Smooth prediction
history.append(final)
final = max(set(history), key=history.count)

# Label + Color
if final == "OT":
    text = f"{vehicle_name} | THREAT ({conf:.2f})"
    color = (0, 0, 255) # RED
else:
    text = f"{vehicle_name} | SAFE ({conf:.2f})"
    color = (0, 255, 0) # GREEN

thickness = 3 if final == "OT" else 2

else:
    # Non-load vehicles
    text = f"{vehicle_name} ({conf:.2f})"
    color = (255, 255, 0) # YELLOW
    thickness = 2

# =====
# DRAW
# =====
cv2.rectangle(frame, (x1, y1), (x2, y2), color, thickness)
cv2.putText(frame, text, (x1, y1 - 10),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.6, color, 2)

cv2.imshow("Real-Time AI System", frame)

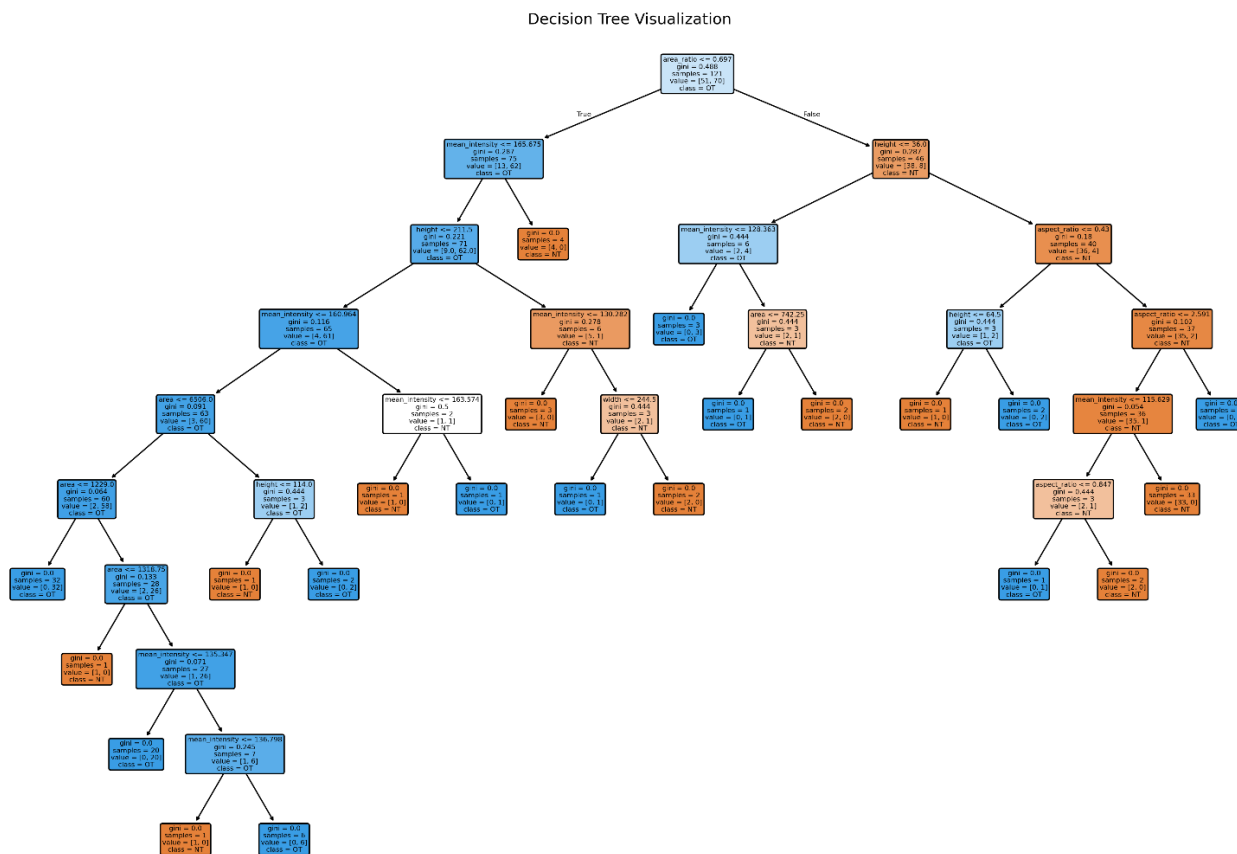
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

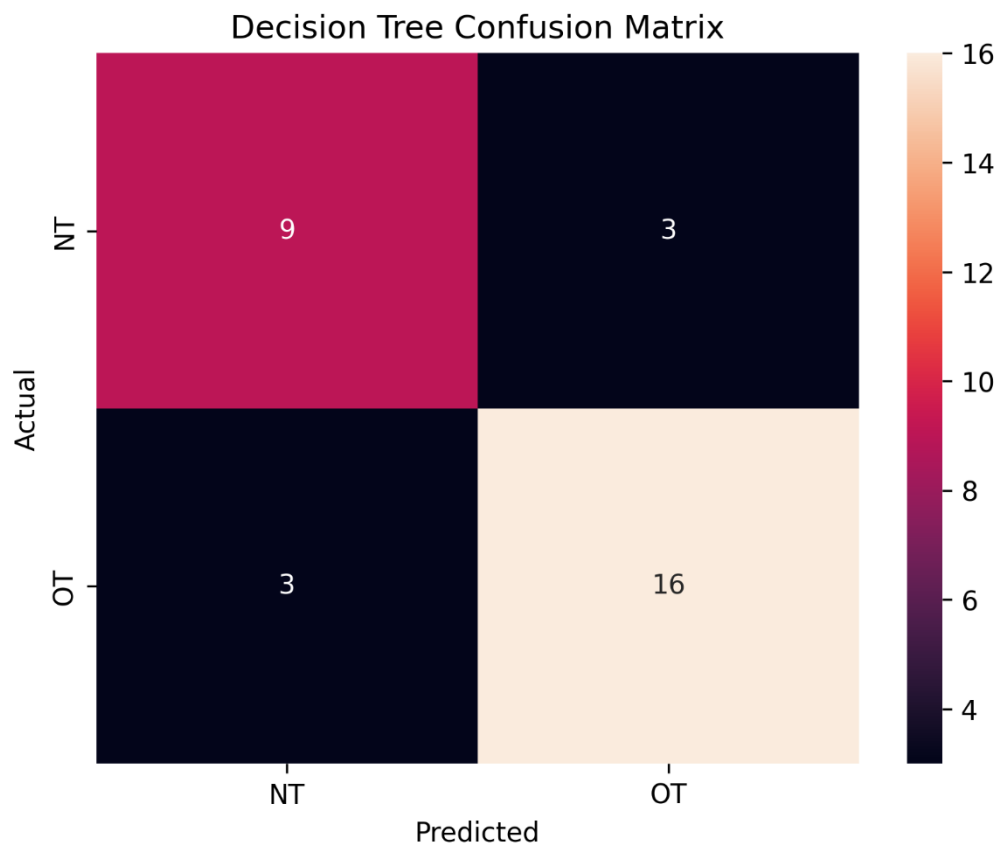
```

APPENDIX II:

Decision Tree Visualization:



Decision Tree CONFUSION MATRIX:



Decision Tree Results:

Confusion Matrix:

```
[[ 9  3]
 [ 3 16]]
```

Decision Tree Feature Importance:

width: 0.0226

height: 0.2129

area: 0.0642

area_ratio: 0.4118

aspect_ratio: 0.0891

perimeter: 0.0000

mean_intensity: 0.1995

Random Forest Results:

===== RANDOM FOREST =====

Accuracy: 0.8064516129032258

Classification Report:

	precision	recall	f1-score	support
NT	0.75	0.75	0.75	12
OT	0.84	0.84	0.84	19
accuracy			0.81	31
macro avg	0.80	0.80	0.80	31
weighted avg	0.81	0.81	0.81	31

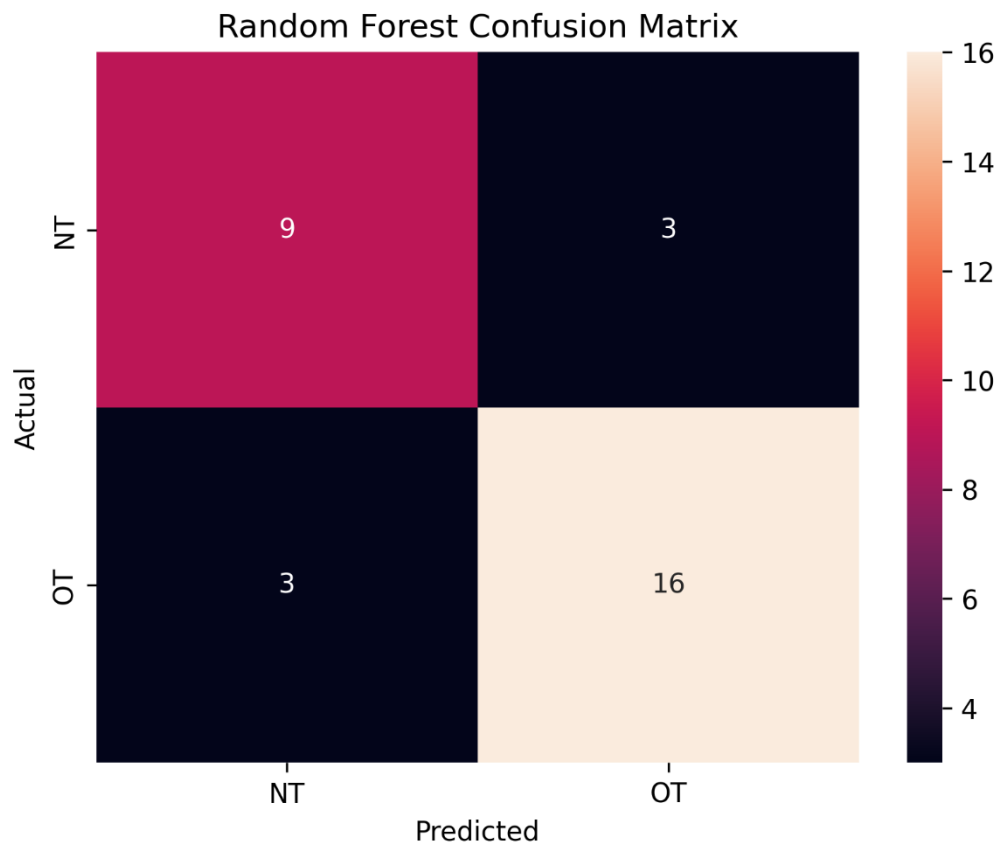
===== DECISION TREE =====

Accuracy: 0.8064516129032258

Classification Report:

	precision	recall	f1-score	support
NT	0.75	0.75	0.75	12
OT	0.84	0.84	0.84	19
accuracy			0.81	31
macro avg	0.80	0.80	0.80	31
weighted avg	0.81	0.81	0.81	31

Random Forest Confusion Matrix:



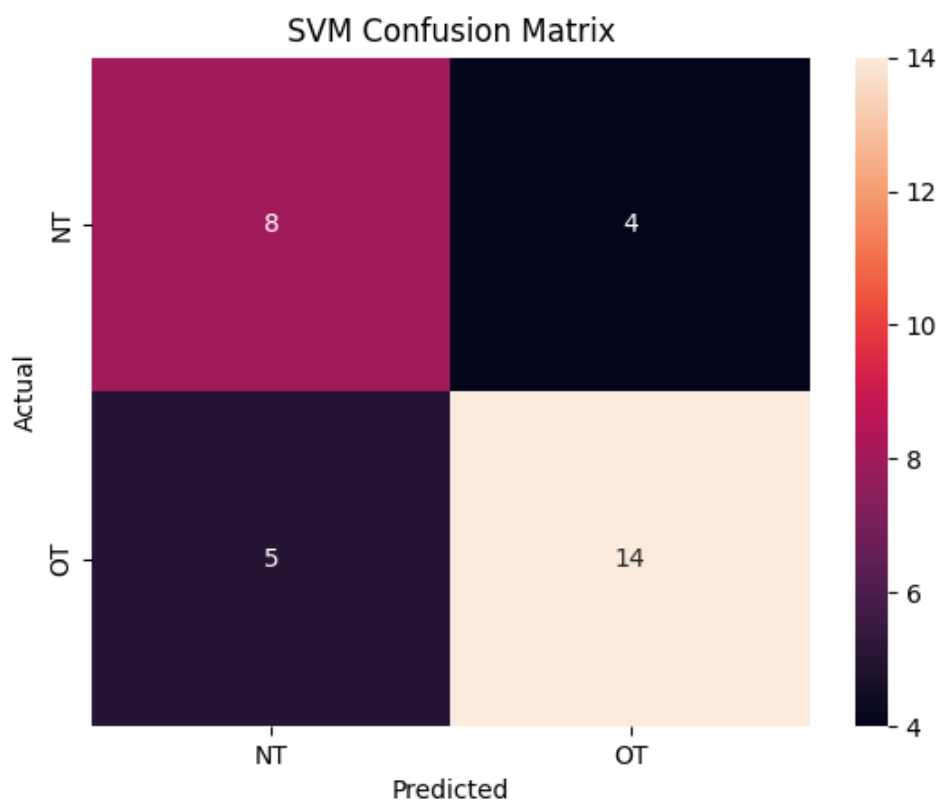
SVM Results:

Classification Report:

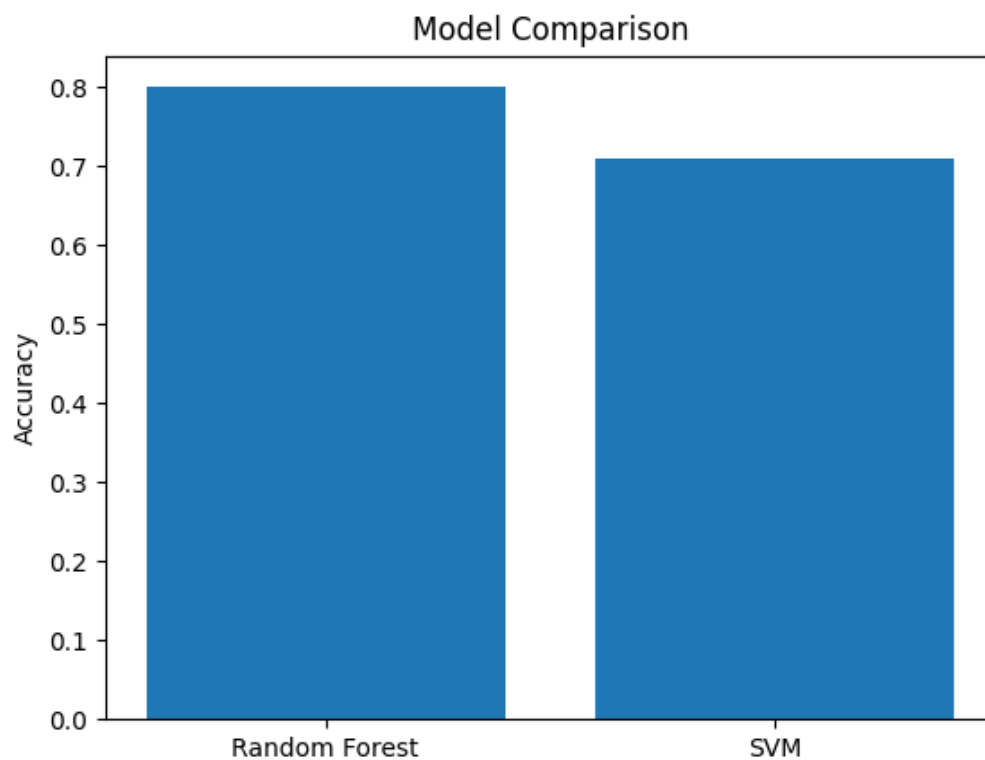
	precision	recall	f1-score	support
NT	0.62	0.67	0.64	12
OT	0.78	0.74	0.76	19
accuracy			0.71	31
macro avg	0.70	0.70	0.70	31
weighted avg	0.71	0.71	0.71	31

SVM model and scaler saved successfully!

SVM CONFUSION MATRIX:



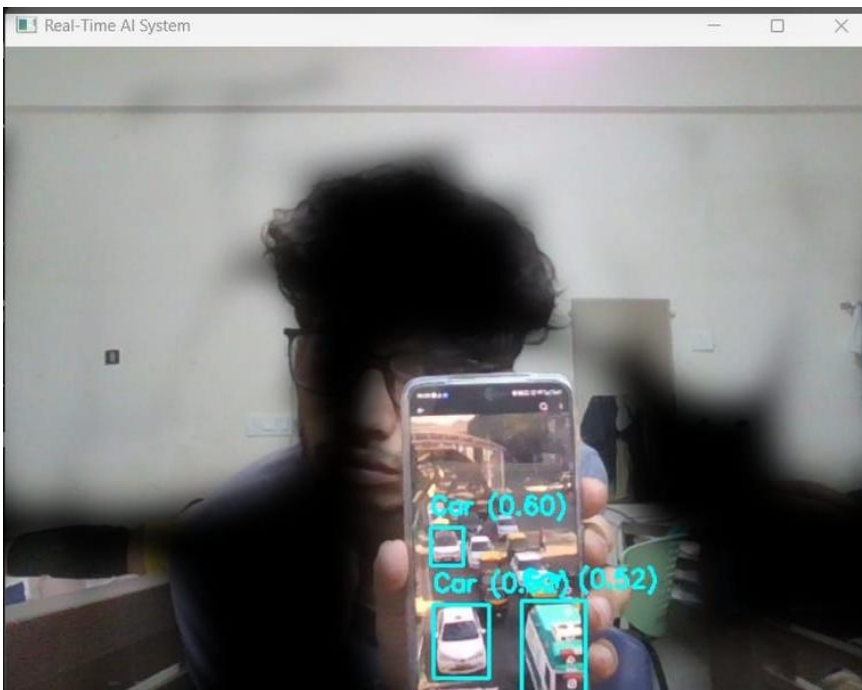
Model Comparison:



Detection Test:



Real Time Vehicle Type detection:



Real Time Overload detection:

